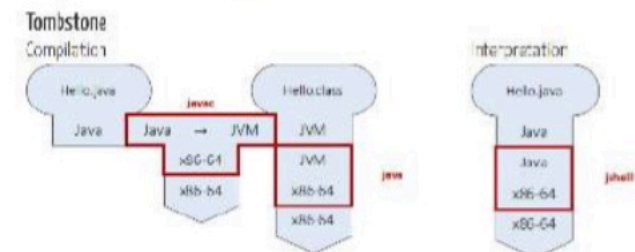
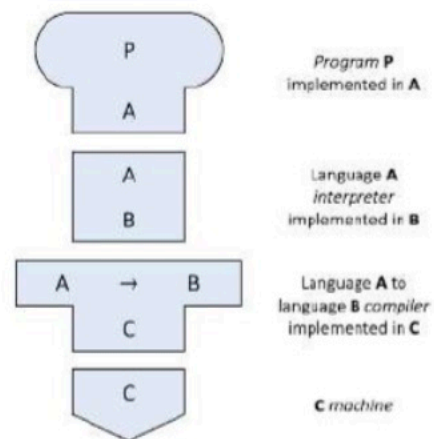


## Programming Languages

- Dynamic v/s Static Typing: Dynamic languages have variables that can hold values of multiple different unrelated types. Static languages have variable types that must be declared and cannot be changed
- Strong v/s Weak Typing: Stronger languages have greater rules in their type system to ensure type safety

## Tombstone Diagram



## Subtyping

- T is a subtype of S ( $T <: S$ ) if code written for S can be safely used for T
- T has a narrower scope, while S has a wider scope
  - T can be put into S (widening)
  - S can be explicitly typecast into T (narrowing)
- Properties
  - Reflexive:  $T <: T$
  - Transitive:  $T <: S, S <: U, T <: U$
  - Anti-Symmetric: If  $T <: S, S <: T$ , then  $S = T$

## Primitive Types

- byte <: short <: int <: long <: float <: double
- char <: int

## Reference Types

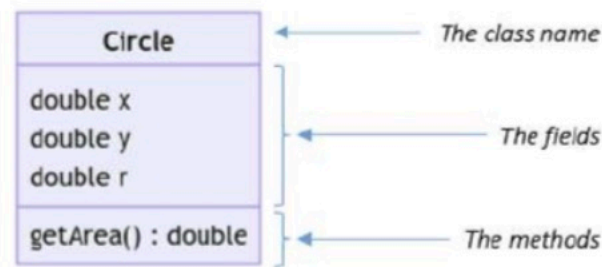
- Anything that is not a Primitive Type such as classes
- Stores only a reference to the value
- Two reference variables can share the same value like pointers, equality by default compares references not actual value
- If uninitialized, have null values and throw errors

## Class Fields and Methods

- Use the **static** keyword to specify
- Fields can be accessed using Class.field, common value shared across the entire class
- Methods can also be accessed using Class.method(), but the **this** keyword will **NOT** work

## Pillars of OOP

- Encapsulation: Bundle of variables (fields) and functions (methods), by making a **class**, that can be represented in a class diagram



- Abstraction: Writing reusable code by grouping sets of instructions
- (Not a Pillar) Composition: Creating wrapper class around an object with additional fields, models a **Has-A** relationship
- Inheritance: Preserve methods and fields of the original object, models a **Is-A** relationship
- Polymorphism: Allowing one variable to take on different run-time types, or different methods to be called

## OOP Style

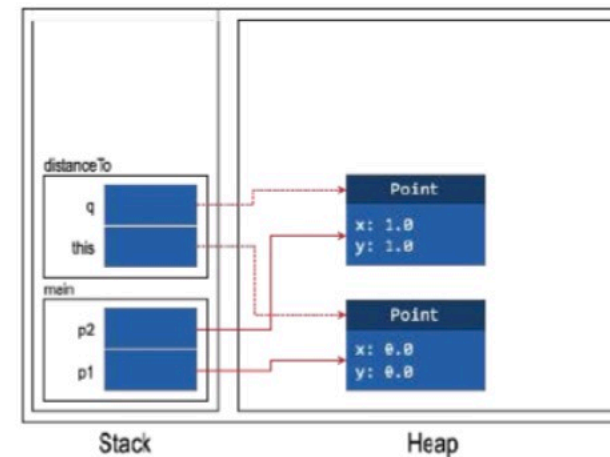
- Information Hiding:
  - Prefer marking fields as **private**, unless necessary, to keep abstraction barrier
  - Private fields can still be accessed by objects of the same class
  - If absolutely necessary, use a **getter** or **setter** to access
- Tell Don't Ask: Client should tell the class what to do, rather than gathering information from the class for actions

## Stack and Heap

- Stack:
  - Where all variables are allocated and stored
  - Contains **Call Frames**, which are created when methods are invoked and destroyed when methods are returned from
- Heap:
  - Where objects are allocated and stored
  - Whenever **new** is used, a new object is created on the heap
  - Objects are stored as Class Name, Instance Fields and Values, and Captured Variables
- Example Diagram when move is called

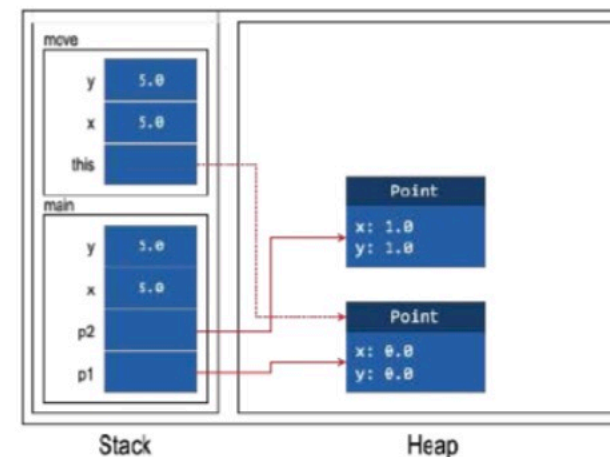
```
class Point {
    private double x;
    private double y;
    public Point(double x, double y) {}
}
```

```
public double distanceTo(Point q) {}
}
Point p1 = new Point(0, 0);
Point p2 = new Point(1, 1);
p1.distanceTo(p2);
```



- Example Diagram when distanceTo is called

```
class Point {
    // ...
    public void move(double x, double y) {}
}
Point p1 = new Point(0, 0);
Point p2 = new Point(1, 1);
double x = 5;
double y = 5;
p1.move(x, y);
```



## Inheritance

- Use the **extends** keyword
- Use **super()** to call the parent constructor, **MUST BE FIRST LINE**
- Overriding
  - Use the **@Override** annotation
  - Must match **Method Descriptor** (Name of method, Type of Parameters, Return Type)
- Overloading
  - Method with same name but different **Method Signature** (Name of method, Type of Parameters)

- Liskov Substitution Principle
  - Any property of objects of type T should be true for any object of type S where  $S <: T$
  - Don't break expectations of the parent class
  - Can use the **final** keyword to prevent inheritance of classes, or modification of methods, or re-assignment of fields

## Dynamic Binding

- Compile Time Type v/s Run Time Type
  - Circle c = **new** ColouredCircle()
  - CTT(c) = Circle, RTT(c) = ColouredCircle
- Example: obj.foo(arg)
- Compile Time
  - Check CTT(obj)
  - Check CTT(arg)
  - Find all accessible methods named foo, including those in supertypes of CTT(obj)
  - Find most specific method (narrowest) that fits with CTT(arg)
  - Record the method descriptor
- Run Time
  - Retrieve method descriptor
  - Determine RTT(obj)
  - Start from RTT(obj) and find first method fitting descriptor
  - Recurse upwards until found
- Does not apply to Class methods

## Abstract Classes

- A general class, that should **NEVER** be instantiated
- Use the **abstract** keyword
- Abstract arrays can still be defined
- Abstract methods do not have any body
- A class with abstract methods must be made abstract

## Interface

- Models requirements of classes
- Use the **implements** keyword for inheritance, methods are public abstract by default
- A class can implement multiple interfaces
- Solves Diamond Problem:
  - If inheritance from multiple classes were allowed, there could be conflicting method declarations
  - Java does not know which method to call
  - Using pure interfaces, they can both be satisfied at once
- Casting to interfaces is allowed, since there is a possibility for the class to satisfy the interface even without implementing it
- Can be impure, when methods have method bodies using the **default** keyword

## Wrapper Classes

- Java provides wrapper classes that encapsulate primitive types
- Auto-boxing: Automatically put primitive types into wrapper class (line 1)
- Auto-unboxing: Automatically put value of wrapper class into primitive variable (line 2)